EDITH AML Intelligence Platform: Master Operationalization & Workflow Division Plan

Document Reference: `edith_end_to_end_workflow_plan.md`
Status: PROPOSED & READY FOR EXECUTION
Target Architecture: Modular Monolith in Development / Distributed Microservices in Production
Runtime: .NET 10 LTS, React 19, SignalR WebSockets, PostgreSQL 17 / InMemory Fallback, Vite

1. Executive Assessment: Why the Solution Is Currently Non-Functional

A systematic audit of the solution (`Edith.slnx`) reveals why none of the platform workflows are currently functional end-to-end:

Root Causes

1. **Disconnected Class Libraries (No HTTP Hosting):**
   - Core domain capabilities (`Edith.Cases`, `Edith.Rules`, `Edith.Addons`, `Edith.Screening.Bulk`) were authored as .NET class libraries. While they have unit tests, **no executable Web API hosts them**.
   - The Gateway (`Edith.Gateway`) only reverse-proxies `/api/v1/screen` to `Edith.Screening.Api`. Any request to `/api/v1/cases`, `/api/v1/rules`, `/api/v1/bulk`, or `/api/v1/addons` returns `404 Not Found`.
2. **SignalR WebSocket Disconnect:**
   - `CaseHub` exists in `Edith.Cases`, but is not registered or mapped to any endpoint in `Edith.Gateway`. Real-time lock collision broadcasting and live progress streaming cannot operate.
3. **Database Availability Dependency:**
   - If PostgreSQL (Docker Compose) is not running locally, database migrations and state queries fail, leaving the system inoperable unless an in-memory dual-mode persistence layer is available.

4. **Port Binding Conflicts:**
    - Launch profiles configured for port `5000` crash when local background processes (or IIS/Windows services) hold port `5000`.
5. **Frontend Mock Masking:**
    - The React frontend falls back silently to static mock objects when API requests fail, masking backend connectivity errors rather than surfacing actionable diagnostics.

2. Solution Division Strategy: The Modular Architecture

To enable rapid local development while preserving the production Kubernetes microservices target, we divide the solution into **7 Functional Workflows** backed by a **Dual-Mode Hosting Strategy**:

```
+-------------------------------------------------------------------------------------------
----------+
|                                   SOLUTION DIVISION TOPOLOGY
|
+--------------------------+--------------------------------+-----------------------------
----------+
| Mode                     | Hosting Architecture           | Target Usage
|
+--------------------------+--------------------------------+-----------------------------
----------+
| Development (Zero Dep)   | Unified Modular API Host       | Visual Studio (F5), single
port,       |
|                          | (Edith.Gateway hosts all       | InMemory seeded stores, zero
Docker    |
|                          | modules & SignalR Hubs)        | dependencies required to boot
|
+--------------------------+--------------------------------+-----------------------------
----------+
| Production (Enterprise)  | Distributed Container Mesh     | Docker Compose / Kubernetes
(Helm),    |
|                          | (YARP Gateway -> 4 Microservices| PostgreSQL 17, OpenSearch
2.x, Kafka, |
|                          | with strict mTLS & Kafka)      | Patroni HA
|
+--------------------------+--------------------------------+-----------------------------
----------+
```

Clean Module Separation

- `Edith.Gateway`: Entry point, JWT authentication, tenant resolution, unified endpoint registration in Dev, YARP proxy in Prod.
- `Edith.Screening.Api` / `Edith.Screening.Core`: Real-time transaction & sanctions screening, SIMD tokenization, phonetic matching, whitelist filtering.
- `Edith.Screening.Bulk`: Asynchronous chunked CSV/XML parsing, Kafka streaming, delta rescreening sync.
- `Edith.Cases`: 3-pane case management, advisory collision locking, state machine validation, SignalR `CaseHub`.
- `Edith.Rules`: Visual rule builder execution, condition evaluation, historical backtesting sandbox.
- `Edith.Addons`: UAE goAML STR/CTR XML generation, GenAI investigative narrative copilot, Cytoscape network graph, Teams Adaptive Cards.
- `Edith.Database.Migrator`: Centralized PostgreSQL 17 schema migrations and seed scripts.
- `src/Web/edith-portal`: React 19 SPA with global command palette, user profile drawer, and real-time SignalR connectivity.

3. The 7 End-to-End Functional Workflows

Detailed Functional Breakdown

Workflow ID	Name & Scope	UI Screens	Key Endpoints	Supporting Backend Service
WF-1	**Identity, Auth & Entitlements**	App Shell, Profile Drawer, Page 13 (IAM)	`POST /api/v1/iam/login` `GET /api/v1/iam/entitlements` `POST /api/v1/iam/switch-role`	`Edith.Gateway.Services.EntitlementsService`, `InMemoryTenantRegistry`
WF-2	**Real-Time Transaction Screening**	Page 2 (Single Screening), Page 6 (Whitelist)	`POST /api/v1/screen/transaction` `GET /api/v1/whitelists` `POST /api/v1/whitelists`	`Edith.Screening.Core.Engine.ScreeningEngine`, `InMemoryWhitelistService`
WF-3	**Bulk File Screening & Delta Sync**	Page 3 (Bulk Screening), Page 14 (Delta Rescreening)	`POST /api/v1/bulk/jobs` `GET /api/v1/bulk/jobs/{id}` `POST /api/v1/delta/sync` `POST /api/v1/delta/authorize`	`Edith.Screening.Bulk.Processing.BulkScreeningProcessor`, `DeltaRescreeningService`, `JobHub`
WF-4	**3-Pane Case Investigation & Locks**	Page 1 (Dashboard), Page 4 (Case Workspace)	`GET /api/v1/cases` `GET /api/v1/cases/{id}` `POST /api/v1/cases/{id}/acquire-lock` `POST /api/v1/cases/{id}/release-lock` `POST /api/v1/cases/{id}/transition` `POST /api/v1/cases`	`Edith.Cases.Services.CaseService`, `CaseLockManager`, `CaseStateMachine`, `CaseHub`

Workflow ID	Name & Scope	UI Screens	Key Endpoints	Supporting Backend Service
			`/{id}/comments`	
WF-5	**Customer KYC 360 & Entity Graph**	Page 5 (KYC 360)	`GET /api/v1/kyc/{id}` `GET /api/v1/kyc/{id}/graph` `GET /api/v1/kyc/{id}/documents`	`Edith.Addons.Graph.EntityGraphVisualizer`, `KycCustomerService`
WF-6	**AML Rule Builder & Simulation Sandbox**	Page 7 (Rule Builder), Page 11 (Sandbox), Page 12 (Settings)	`GET /api/v1/rules` `POST /api/v1/rules` `POST /api/v1/rules/backtest` `GET /api/v1/settings/masters`	`Edith.Rules.Engine.RuleEngine`, `RuleBacktester`, `RuleRepository`
WF-7	**Regulatory goAML, AI Copilot & WORM Audit**	Page 8 (Reports), Page 9 (goAML), Page 10 (Notifications)	`POST /api/v1/addons/goaml/export` `POST /api/v1/addons/ai/narrative` `GET /api/v1/reports/audit` `GET /api/v1/notifications/rules`	`Edith.Addons.GoAml.GoAmlXmlGenerator`, `StrNarrativeCopilot`, `DynamicNotificationEngine`

4. Implementation Plan: 5 Execution Phases

```
⚠️ Failed to render Mermaid diagram: Invalid mermaid header: "gantt". Expected "graph TD", "flowchart LR", "stateDiagram-v2", etc.. Supported types: flowchart/graph, stateDiagram-v2, sequenceDiagram, classDiagram, erDiagram, xychart-beta.
gantt
    title EDITH Workflow Operationalization Roadmap
    dateFormat  YYYY-MM-DD
    section Phase 1: Gateway & Host
    Unified Endpoint Hosting        :p1_1, 2026-10-15, 1d
    SignalR CaseHub & JobHub Mapping:p1_2, after p1_1, 1d
    section Phase 2: Domain Services
    Wire Cases, Rules & Addons APIs :p2_1, after p1_2, 2d
    Wire Bulk & Delta Backgrounders  :p2_2, after p2_1, 1d
    section Phase 3: In-Memory Stores
    Dual-Mode In-Memory Seed Stores :p3_1, after p2_2, 1d
    section Phase 4: Frontend Bridge
    Vite Proxy & Live SignalR Hook  :p4_1, after p3_1, 1d
    Connect 14 Screens to Real APIs :p4_2, after p4_1, 2d
    section Phase 5: Verification
    End-to-End Workflow Validation  :p5_1, after p4_2, 1d
    Performance & SLA Verification  :p5_2, after p5_1, 1d
```

 Mermaid diagram

Phase 1: Unified API Hosting & SignalR Integration in Gateway

1. **Host All Minimal API Modules in `Edith.Gateway`:**
    - Reference `Edith.Cases`, `Edith.Rules`, `Edith.Addons`, `Edith.Screening.Bulk`, and `Edith.Screening.Core` in `Edith.Gateway.csproj`.
    - Create clean route extension groups:
        - `app.MapCaseEndpoints();`
        - `app.MapRuleEndpoints();`
        - `app.MapBulkScreeningEndpoints();`
        - `app.MapAddonEndpoints();`
        - `app.MapKycEndpoints();`
        - `app.MapSettingsEndpoints();`
2. **SignalR WebSocket Hub Endpoints:**
    - Map `/hubs/cases` to `CaseHub`.
    - Map `/hubs/jobs` to `JobHub` (for bulk progress streaming).

- Configure CORS to permit Vite dev origin (`http://localhost:3000`).

Phase 2: In-Memory Domain Services & Dual-Mode Persistence

1. Provide robust in-memory seeded services that mirror PostgreSQL/OpenSearch data structures when running standalone:
   - `InMemoryCaseRepository` with 10 pre-loaded realistic GCC sanctions/PEP cases.
   - `InMemoryRuleRepository` with default FATF high-risk corridor and smurfing rules.
   - `InMemoryKycRepository` with complete customer profiles, transactions, and entity graph linkages.
   - `InMemoryAuditLedger` providing SHA-256 chained audit entries.
2. Ensure services automatically detect PostgreSQL connection string availability:
   - If PostgreSQL connection succeeds -> query database tables.
   - If PostgreSQL connection fails -> seamlessly fall back to seeded memory store without throwing fatal boot exceptions.

Phase 3: Asynchronous Bulk Screening & Background Streaming

1. Register `IBulkScreeningProcessor` and `IDeltaRescreeningService` as background services.
2. When the user uploads a CSV in Page 3:
   - Endpoint returns `HTTP 202 Accepted` with `jobId` .
   - Background worker processes records in chunks and emits progress percentage over SignalR (`JobHub`).
   - Frontend displays active job pill in header and real-time progress bar.

Phase 4: Frontend API Bridge & SignalR Client Hook

1. Update `src/Web/edith-portal/src/services/api.ts` :
   - Replace static mock fallbacks with real HTTP calls to `/api/v1/...` .
   - Surface genuine error toasts when requests fail rather than silently falling back.
2. Establish real SignalR client connections in `App.tsx` :
   - Connect to `/hubs/cases` to receive instant lock updates and status changes.
   - Connect to `/hubs/jobs` to receive real-time bulk screening notifications.

Phase 5: Port Configuration & Single-Click Run Experience

1. **Port Allocation:**

- Gateway/Unified API: `http://localhost:5000` (with automatic fallback to `5050` if 5000 is occupied).
- React Portal: `http://localhost:3000` (proxies `/api` and `/hubs` to Gateway).

2. **Launch Configuration:**
   - Update `Edith.slnLaunch.user` and launchSettings so starting `Edith.Gateway` boots all backend capabilities.
   - Add simple `npm run dev` in `src/Web/edith-portal` for immediate frontend interaction.

5. Verification & Acceptance Criteria Matrix

Workflow	Test Scenario	Acceptance Criteria
WF-1: Auth & IAM	Switch role to `MLRO` in profile drawer	Entitlements update immediately; MLRO-only action buttons appear.
WF-2: Screening	Screen "Viktor Bout" with USD 450,000	Match score >= 94%, decision `BLOCKED`, evaluation latency < 50ms in dev.
WF-3: Bulk Engine	Upload test CSV with 500 rows	Returns `202 Accepted`; live progress bar increments to 100% via SignalR.
WF-4: Case Workspace	Acquire lock on `EDITH-CAS-2026-0042`	Lock badge turns violet; second browser tab is blocked from concurrent edits; transition from `Open` to `UnderReview` succeeds.
WF-5: KYC & Graph	Open Customer KYC 360 for high-risk entity	90-day transactions load; Cytoscape interactive graph renders counterparty links.
WF-6: Rules & Sandbox	Adjust threshold slider to $50,000 and run backtest	Confusion matrix updates; projected false positive percentage recalculates.
WF-7: goAML & Copilot	Click "Generate goAML XML" & "AI Copilot"	Valid CBUAE XML document generates; 5-section STR investigative narrative streams in.

6. Key Decisions & User Guidance

> IMPORTANT
>
> **Key Architectural Decision:** Rather than forcing the local developer machine to juggle 5 distinct web server processes (which triggers port conflicts and excessive memory consumption), we

recommend **hosting all module endpoints and SignalR hubs directly within `Edith.Gateway` during Development (Option B: Modular Host)**. In production, Docker Compose and Kubernetes Helm charts continue to deploy them as separated, isolated microservice containers.

> **TIP**
>
> **Zero-Dependency Boot:** By incorporating in-memory seeded fallbacks for all domain services, the entire platform can be launched and verified immediately in Visual Studio and `npm run dev` even if PostgreSQL and Docker are temporarily offline.